



Algoritmul lui Euclid | Exponențiere rapidă

Andrei Mihai Ciobanu

Algolymp Summer School

Algoritmul lui Euclid

Algoritmul lui Euclid extins

Exponențiere rapidă

Bonus — Implementări recursive

Recapitulare

Resurse

Probleme de antrenament

Algoritmul lui Euclid

Cel mai mare divizor comun

Cel mai mare divizor comun

$\text{cmmdc}(a, b)$ = cel mai mare număr natural d astfel încât $d \mid a$ și $d \mid b$. Se mai notează și $\text{gcd}(a, b)$ (*greatest common divisor*) sau doar (a, b) .

Cel mai mic multiplu comun

$\text{cmmmc}(a, b)$ = cel mai mic număr natural m astfel încât $a \mid m$ și $b \mid m$. Se mai notează și $\text{lcm}(a, b)$ (*lowest common multiple*) sau doar $[a, b]$.

Exemple:

- $\text{gcd}(12, 8) = 4$ $\text{gcd}(17, 5) = 1$ $\text{gcd}(100, 75) = 25$
- $\text{lcm}(4, 6) = 12$ $\text{lcm}(3, 5) = 15$

Proprietăți (I)

- $\gcd(a, 0) = \gcd(a, a) = |a|$ (cazuri de bază)
- $\gcd(a, b) = \gcd(b, a)$ (comutativitate)
- $\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$ (asociativitate)
- $\gcd(k \cdot a, k \cdot b) = k \cdot \gcd(a, b)$, pentru $k > 0$
- dacă $\gcd(a, b) = d$, atunci $\gcd\left(\frac{a}{d}, \frac{b}{d}\right) = 1$
- $a \cdot b = \gcd(a, b) \cdot \text{lcm}(a, b)$

Proprietăți (II)

- $\gcd(a, b) = \gcd(a \pm k \cdot b, b), \forall k \in \mathbb{N}$
- **Consecință directă:** $\gcd(a, b) = \gcd(b, a \bmod b)$
 $a \bmod b = \text{restul împărțirii lui } a \text{ la } b$
- **Prin descompunere în factori primi:** dacă $n = \prod p_i^{a_i}$ și $m = \prod p_i^{b_i}$, atunci:

$$\gcd(n, m) = \prod p_i^{\min(a_i, b_i)} \quad \text{lcm}(n, m) = \prod p_i^{\max(a_i, b_i)}$$

Exemplu:

$$n = 2^3 \cdot 3^2, m = 2^1 \cdot 3^4 \Rightarrow \gcd = 2^1 \cdot 3^2 = 18, \text{lcm} = 2^3 \cdot 3^4 = 648$$

Algoritmul lui Euclid — Idee

Știm că $\gcd(a, b) = \gcd(a - b, b)$. Putem aplica această regulă repetat până ajungem la cazul de bază $\gcd(a, 0) = a$.

Exemplu: $\gcd(12, 8)$

- $\gcd(12, 8) = \gcd(4, 8) = \gcd(4, 4) = \gcd(0, 4) = 4 \checkmark$

Contraexemplu: $a = 1000, b = 1$

$\gcd(1000, 1) \rightarrow \gcd(999, 1) \rightarrow \gcd(998, 1) \rightarrow \dots \rightarrow \gcd(0, 1) = 1$

1000 de pași pentru un rezultat evident. Putem face mai bine?

Algoritmul lui Euclid — Idee

Observăm că scăderile repetate cu același b sunt echivalente cu restul împărțirii:

$$\gcd(a, b) = \gcd(a - b, b) = \gcd(a - 2b, b) = \dots = \gcd(a \bmod b, b)$$

Observație cheie

$\gcd(a, b) = \gcd(b, a \bmod b)$, cu cazul de bază $\gcd(a, 0) = a$.

Exemplu: $\gcd(48, 18)$

- $\gcd(48, 18) = \gcd(18, 12)$ ($48 \bmod 18 = 12$)
- $\gcd(18, 12) = \gcd(12, 6)$ ($18 \bmod 12 = 6$)
- $\gcd(12, 6) = \gcd(6, 0)$ ($12 \bmod 6 = 0$)
- $\gcd(6, 0) = 6$ ✓

Algoritmul lui Euclid — Implementare iterativă

```
1  int gcd(int a, int b) {  
2      while (b != 0) {  
3          int r = a % b;  
4          a = b;  
5          b = r;  
6      }  
7      return a;  
8  }
```

- Timp: $O(\log \min(a, b))$ Spațiu: $O(1)$
- Exemplu: $\text{gcd}(48, 18) = 6$
- Cel mai mic multiplu comun: $\text{lcm}(a, b) = a / \text{gcd}(a, b) * b$

Tip: împarte înainte de înmulțire pentru a evita overflow

Algoritmul lui Euclid extins

Algoritmul lui Euclid extins — Idee

Identitatea lui Bézout

Pentru orice $a, b \in \mathbb{Z}$, există $x, y \in \mathbb{Z}$ astfel încât:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

Algoritmul lui Euclid extins găsește x și y urmărind coeficienții înapoi prin pașii algoritmului lui Euclid.

Exemplu: $\gcd(35, 15) = 5$

- $35 = 2 \cdot 15 + 5 \Rightarrow 5 = 1 \cdot 35 + (-2) \cdot 15$
- Soluție: $x = 1, y = -2$ ✓

▷ Această secțiune este un concept ușor mai dificil — poți reveni la ea după ce ești confortabil cu restul lecției.

Algoritmul lui Euclid extins — Implementare iterativă

```
1 // Gasește  $x, y$  cu:  $a \cdot x + b \cdot y = \gcd(a, b)$ 
2 int gcd_ext(int a, int b, int &x, int &y) {
3     int x0 = 1, y0 = 0;
4     int x1 = 0, y1 = 1;
5     while (b != 0) {
6         int q = a / b;
7         int xn = x0 - q * x1;
8         int yn = y0 - q * y1;
9         int r = a % b;
10        a = b; b = r;
11        x0 = x1; y0 = y1;
12        x1 = xn; y1 = yn;
13    }
14    x = x0; y = y0;
15    return a;
16 }
```

- `int &x` = parametru prin referință: modificările din funcție se văd la apelant
- Timp: $O(\log \min(a, b))$ Spațiu: $O(1)$

Exponențiere rapidă

Problema

Calculează a^n eficient. Varianta naivă ($a \cdot a \cdots a$, de n ori) are complexitate $O(n)$.

Observație cheie

$$a^n = \begin{cases} 1 & n = 0 \\ (a^{n/2})^2 & n \text{ par} \\ a \cdot a^{n-1} & n \text{ impar} \end{cases}$$

Exemplu: $2^{10} \rightarrow 2^5 \rightarrow 2^4 \rightarrow 2^2 \rightarrow 2^1 \rightarrow 2^0$ (5 pași în loc de 10)

Exponențiere rapidă — Implementare iterativă

```
1 long long power(long long base, int exp) {  
2     long long result = 1;  
3     while (exp > 0) {  
4         if (exp % 2 == 1)  
5             result *= base;  
6         base *= base;  
7         exp /= 2;  
8     }  
9     return result;  
10 }
```

- Timp: $O(\log n)$ Spațiu: $O(1)$
- **Atenție:** `base * base` poate da overflow pentru baze mari — în concursuri se lucrează de obicei modulo un număr prim

Bonus — Implementări recursive

Recursivitate — Ce este?

Definiție

O funcție este **recursivă** dacă se apelează pe ea însăși.

Orice funcție recursivă are nevoie de:

- Un **caz de bază** — condiția de oprire (altfel rulează la infinit)
- Un **apel recursiv** pe o subproblemă mai mică

Exemplu: $n! = n \cdot (n - 1)!$, cu cazul de bază $0! = 1$

▷ *Recursivitate — concept nou; caută “funcții recursive C++” pe cppreference.com sau pbinfo.ro (vezi slide-ul Resurse)*

De ce implementări recursive?

- Codul reflectă direct definiția matematică — mai ușor de înțeles și de scris
- De obicei mai scurt decât varianta iterativă

Atenție — dezavantaje

- Overhead de apeluri de funcție — ușor mai lent
- Pentru intrări foarte mari poate da **stack overflow**

Concluzie

Varianta recursivă e utilă pentru a **înțelege** algoritmul; varianta iterativă e de preferat în concursuri.

Algoritmul lui Euclid — Implementare recursivă

```
1 int gcd(int a, int b) {  
2     if (b == 0)  
3         return a;  
4     return gcd(b, a % b);  
5 }
```

- Codul reflectă direct proprietatea $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$
- Aceeași complexitate ca varianta iterativă: $O(\log \min(a, b))$

Algoritmul lui Euclid extins — Implementare recursivă

```
1  int gcd_ext(int a, int b, int &x, int &y) {
2      if (b == 0) {
3          x = 1; y = 0;
4          return a;
5      }
6      int x1, y1;
7      int d = gcd_ext(b, a % b, x1, y1);
8      x = y1;
9      y = x1 - (a / b) * y1;
10     return d;
11 }
```

- Timp: $O(\log \min(a, b))$ Spațiu: $O(\log \min(a, b))$ — stiva de apeluri

Exponențiere rapidă — Implementare recursivă

```
1 long long power(long long base, int exp) {  
2     if (exp == 0)  
3         return 1;  
4     if (exp % 2 == 0) {  
5         long long half = power(base, exp / 2);  
6         return half * half;  
7     }  
8     return base * power(base, exp - 1);  
9 }
```

- `half * half` evită calculul dublu

`power(base, exp/2) * power(base, exp/2)`

- Timp: $O(\log n)$ Spațiu: $O(\log n)$ — stiva de apeluri

Recapitulare

Recapitulare — Complexități

Algoritm	Timp	Spațiu
gcd iterativ	$O(\log \min(a, b))$	$O(1)$
gcd extins iterativ	$O(\log \min(a, b))$	$O(1)$
gcd recursiv	$O(\log \min(a, b))$	$O(\log \min(a, b))$
gcd extins recursiv	$O(\log \min(a, b))$	$O(\log \min(a, b))$
Exponențiere rapidă iterativă	$O(\log n)$	$O(1)$
Exponențiere rapidă recursivă	$O(\log n)$	$O(\log n)$

Resurse

Algoritmul lui Euclid — gcd și lcm

- cp-algorithms.com — Euclidean algorithm (EN)
- cp-algorithms.com — Extended Euclidean algorithm (EN)

Exponențiere rapidă

- cp-algorithms.com — Binary exponentiation (EN)

Recursivitate

- cppreference.com — documentație funcții C++
- Caută *“recursivitate”* pe pbinfo.ro

Probleme de antrenament

pbinfo.ro

- **cmmdc**
- **cmmmc**
- **putere2**

infoarena.ro

- **euclid3**